SIGma **Fall 2026, November 5th, 2026**

2027 CG:SHOP: multi-robot cover

Lecturer: Ian Chen *Scribe: Ian Chen*

In the previous weeks, we've been focusing on the single-robot case, with either a square-cutter or circle-cutter. There is a very rich space in determining how to split the problem so that we can load-balance across multiple-robots. Namely, as in CG:SHOP 2027, we will consider the makespan objective: minimize the maximum length. In particular, our main tool will be an approximation algorithm for TreeCover: cover a graph using k trees minimizing the maximum size of any tree. See original paper: [4].

problem formulation The terrain consists of "large" cells, each binary (visitable or not), with each large cell a 2×2 grid of small cells. Let n be the number of visitable small cells. There are many variations here that we can consider:

- Weighted (large cells weight $w_i \in \mathcal{Z}^{\geq 1}$) or unweighted (large cells weight 4) cases, where the cost of a move is the average of its start and end.
- Robots are non-blocking (they can occupy the same cell); robots are sized as small cells.
- Each robot is given an initial cell for each robot.
- Without return (paths) or with return (tours).

And as typical, we consider the make-span objective (minimize the max length). Unsurprisingly, the problem is proven to be NP-hard, by reduction from 3-partition (split an array of length $3m$ into m sets, all of equal weight and of size 3) [4].

1 Existing Work

First, we describe the standard approaches based on a single spanning tree, and describe why they are insufficient. Thus, this motivates the new tool in [4]: the forest cover.

spanning tree coverage (STC) STC [2] is for a single robot: take any spanning tree of the large cells, and circumnavigate it. This is optimal in the unweighted case, and in the weighted case is off by an additive term of the largest weight (Fig. 1).

multi-robot spanning tree coverage (MSTC) MSTC [3] improves the path that circumnavigates the spanning tree. Each robot walks the arc from the circumnavigation in the same direction (given start point), say counterclockwise, as in Fig. 2.

1. We can choose start points, without return. Then, it is very easy to subdivide evenly (with small additive error).
2. We can choose start points, with return. Then, it is not easy (say, a 2-approx naively).
3. Fixed start positions (say, unweighted, with k robots). If a stretch is longer than $n/2$, then we have the others assist ($k \geq 3$), as in Fig. 2.

Proposition: The cost is at most $n/2$, which is the best for a tree in the worst case. Indeed, there may be a bad spanning tree that place the robots adjacent in the circumnavigation.

4. Weighted (then stretch the paths accordingly).

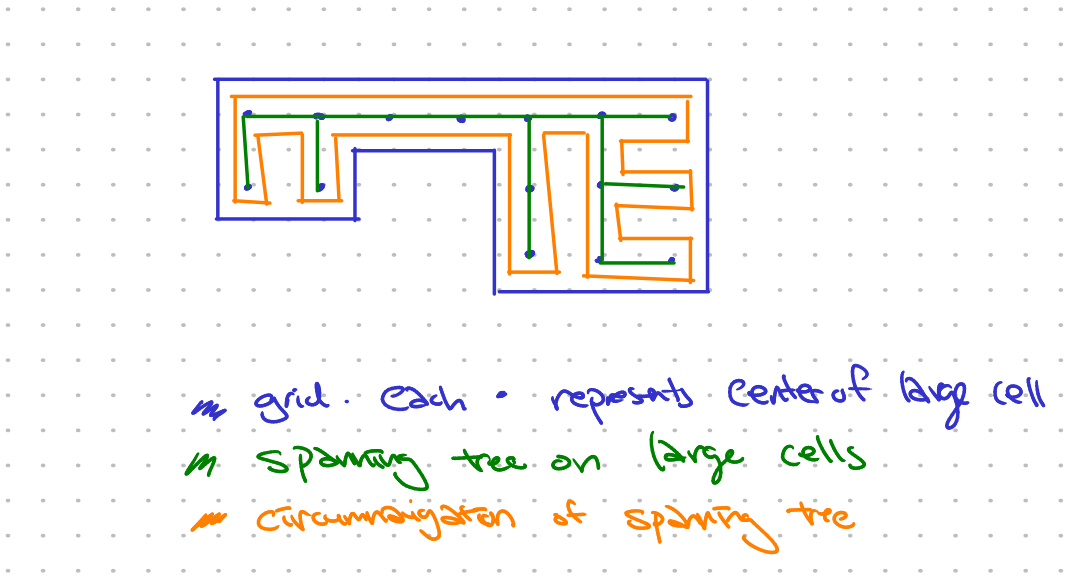


Figure 1: **Spanning tree cover:** On unweighted-graph wirh large-cells, circumnavigation of STC is optimal. Without return, can remove any edge. If weighted, then removing the most expensive edge achieves a small additive approximation error.

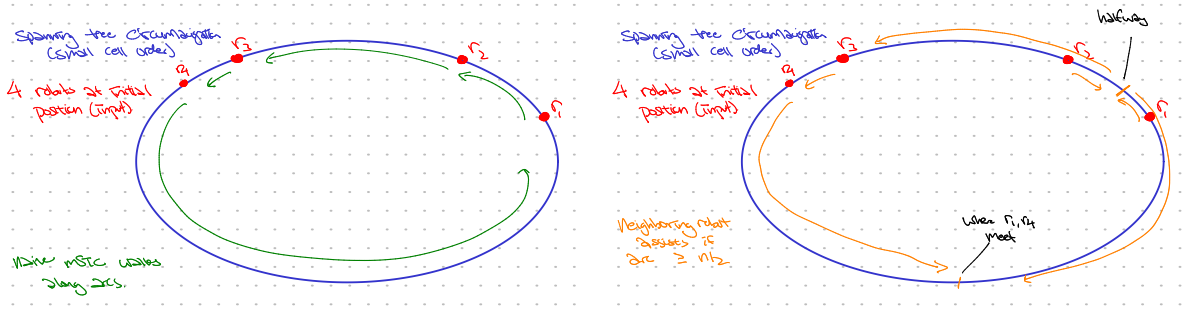


Figure 2: **MSTC:** multi-robot spanning tree cover first finds a spanning tree, and splits it into arcs based on the initial positions of robots. Left: naive mSTC. Right: local assists for long unbalanced arcs.

The mSTC algorithm has no good provable approximation factor (scales according to k). Indeed, a single tree could be bad, putting multiple robots either nearby in the circumnavigation, while a better tree could spread them out evenly. A heuristic could be to try multiple spanning trees, but in general, this is difficult to do well.

2 multi-robot forest cover

Zheng et al. argues that a much more natural approach than sticking with this single spanning tree and circumnavigation approach is to give each robot its own (possibly overlapping) tree, and have them each circumnavigate its smaller tree. This is known as a forest cover, where we take a collection of trees such that their union covers the graph. Again, as in mSTC, we want to cover the large cells (so that circumnavigation is on the small cells).

Indeed, suppose we can solve the optimal forest cover. Then, we immediately would have a 4-approximation to multi-robot lawn mowing (and often a bit better), as the forest cover is shorter than the path cover (as paths are trees), and the cost is $4\times$ the forest cover (number of small cells is $4\times$ the large cells). This is a similar argument to the first lecture on spanning tree approximation for TSP!

Finally, the paper applies prior work for a 4-approx for the forest cover algorithm, leading to a 16-approximation for lawn-mowing. In the weighted case, with $\epsilon = k(w_{\max}/w_{\text{sum}})$, there is a $4(1+\epsilon)$ approx for tree cover and so a $16(1+\epsilon)$-approximation for weighted lawn-mowing.

3 4-approx unweighted tree cover

We conclude these notes with a description of the 4-approx unweighted tree cover [1]; similar for weighted.

- **input**: unweighted graph G, with distinct roots $r_1, \ldots, r_k$
- **output**: trees $T_1, \ldots, T_k$ (not necessarily disjoint), minimizing the maximum size

These roots represent the (large cell at) the start point of each robot, which we assume are distinct. The graph is on the large-cells, and we have grid connections; outputting the trees allows us to circumnavigate efficiently. This is perhaps the most involved algorithm we have discussed so far, and requires some preliminaries on combinatorial optimization.

3.1 aside: max-flow min-cut theorem

We formulate the optimization problem on a digraph G with $V = \{s,t\} \sqcup V'$, where s is the source and t is the sink, and with edge capacity c. A flow assigns $f : E \to [0, c(e)]$ to each edge.
Proposition: Any flow $\leq$ any cut (weak duality).
Proof: Consider the cut $(S, \bar{S})$. The value of the cut is the sum of all capacities through it, for which the flow cannot exceed (as flow on each edge is bounded by the capacity).

The strengthening, i.e. some flow has equal value to the cut, is known as strong duality, and is much more interesting. Consider for each edge $u \to v$ the *residual edge* $v \to u$, where

$$w(u \to v) = c(u \to v) - f(u \to v)$$
$$w(v \to u) = f(u \to v)$$

See Fig. 3 for an example. That is, this residual weight is the amount of flow we can change the flow by; if we have positive flow, we can decrease it by that amount, or increase it up to the capacity.

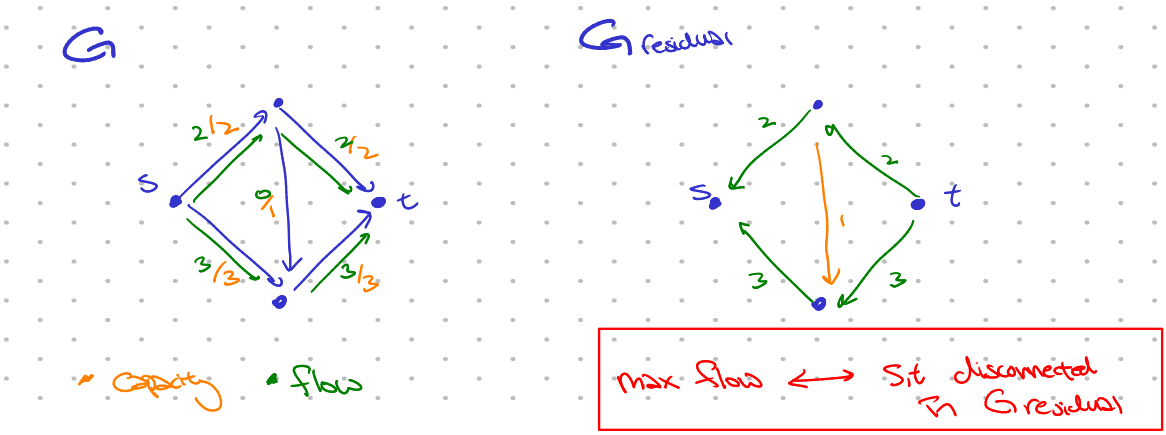


Figure 3: **Residual graph:** A graph with 4 vertices and max-flow of 5; residual graph has no $s-t$ path.

Proposition: Max flow $\implies$ no s,t path in G'.

Therefore, we have an (s,t)-cut, the value is the same, and we achieve that max-flow = min-cut. Flows are very useful in many graph problems, e.g. matchings/scheduling.

> **Exercise 3.1:** Why can we assume that there is a max-flow using only integers, when the capacities are integers?

Corollary: There exists an L-matching in a bipartite graph $(L \cup R, E)$ with $|L| \leq |R|$ if $\forall S \subseteq L, |N(S)| \geq |S|$.
Proof: Consider the graph H with a source s connected to all of L, and a sink t connected to all of R, each with edge capacity 1. We assign edges in the original bipartite graph a capacity $|L|$. Then, there exists a min-cut $(\{s\}, L \cup R \cup \{t\})$, because replacing $S \cap L$ and moving that and its neighbors to T does not increase the cut.

3.2 the approx algorithm

Now, we are ready to give the approx algorithm. Namely, it is sensible to:

1. start from a forest
2. break apart "large" trees, and move them to another root

To determine "large", we parameterize based on a parameter $B \in [\![|V|]\!]$ (where we say $|T|$ is number of vertices). Precisely, we break the smallest branch, i.e. a vertex v and some of its child subtrees, where the branch has size $> B$, and contract into v (Fig. 4).

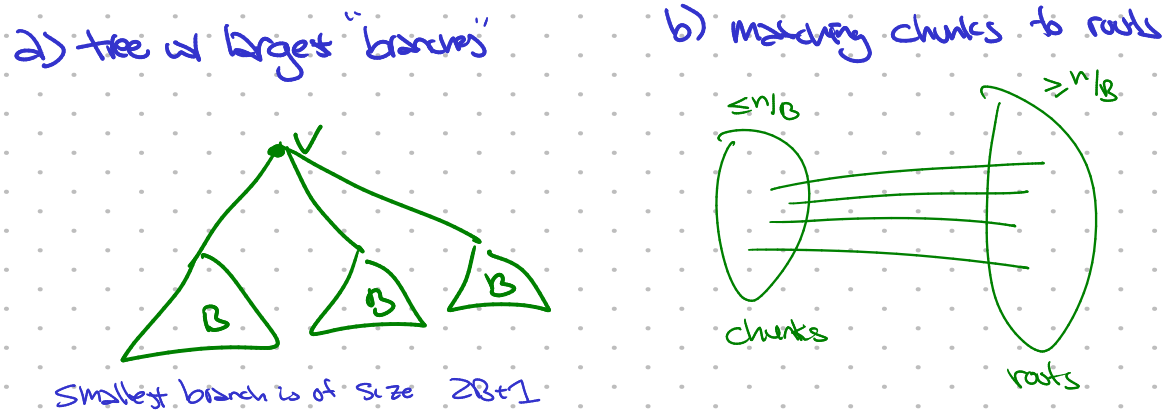


Figure 4: **Stages of TreeCover:** a) chunks are bounded by $2B + 1$ b) matching chunks to roots

We take a min-max matching of the chunks to the roots, where the weight is the distance from the chunk to the root (Fig. 4). Then, the total cost is

$$\max_{i \in [\![k]\!]} \{|\text{leftover } i| + |\text{chunk matched to } i| + \text{dist}(i)\}$$

Observe that the number of chunks is at most $|V|/B$, so we should set $B \geq |V|/k$ to ensure such a matching exists. Finally, we need to choose B and to bound the cost. For this, we can simply try every B (a linear search) and take the one with the minimum cost. In particular, for the analysis, the algorithm is no worse than setting $B = B^*$, the optimal.

$$\max_{i \in [\![k]\!]} \{\ldots\} \leq \underbrace{\max(\text{leftover})}_{\leq B^*} + \underbrace{\max(\text{chunk})}_{\leq 2B^*+1} + \underbrace{\max(\text{dist in matching})}_{\leq B^*-1} \leq 4B^*$$

This holds as, restricting to only edges with weight $\leq B^* - 1$, each subset S (of chunks) has at least $|S|$ neighbors (pigeonhole: consider the roots covering S in the optimal). Therefore, we can apply Hall's theorem (as described above) to show there is a matching here; if S has at least $|S|$ neighbors for every S subset of roots, then there is a perfect matching.

solutions

> **Solution to Exercise 3.1:** We can consider an iterative process for finding a max-flow by finding maximal paths in the residual graph. Inductively, as flows are integral, then so are residuals, and so flows stay integral. In general, this is known as the Ford-Fulkerson algorithm.

References

[1] G. Even et al. “Min-max tree covers of graphs”. In: *Operations Research Letters* 32 (2004), pp. 309–315.

[2] Y. Gabriely and E. Rimon. “Spanning-tree based coverage of continuous areas by a mobile robot”. In: *Annals of Mathematics and Artificial Intelligence* 31 (2001), pp. 77–98.

[3] N. Hazon and G. Kaminka. “Redundancy, efficiency, and robustness in multi-robot coverage”. In: *Proceedings of the International Conference on Robotics and Automation*. 2005, pp. 735–741.

[4] Xiaoming Zheng et al. “Multirobot Forest Coverage for Weighted and Unweighted Terrain”. In: *IEEE Transactions on Robotics* 26.6 (2010), pp. 1018–1031. DOI: 10.1109/TRO.2010.2072271.